

JobCompass Individual Reflection Draft

Status: AI-assisted draft for personal revision, not a verified declaration of completed student work.

Group: 11. Student name and student ID must be completed by the author. The group members are OU JUNHANG, ZHANG XIQIAN, JIANG YIFAN, CAI SHANGHENG, GUO ZHENGKUN. Every member must replace the contribution section with evidence of their own work before submission.

Introduction

JobCompass is a local Python application for comparing a candidate's resume with jobs collected from multiple sources. The proposed problem is practical: applicants often repeat the same search across several websites, interpret inconsistent job descriptions and lose track of applications. The project combines resume extraction, job normalization, explainable ranking and an application board in one interface. It also offers optional DeepSeek assistance for identifying evidence in a resume and improving its wording. The program is intended to help a person organize decisions, rather than predict whether an employer will hire that person.

The original request allowed generative AI to assist with implementation. Codex produced the initial architecture, source code, tests, interface and documentation in response to that request. This draft therefore does not claim that a student personally wrote or independently reviewed those components. The final individual report must distinguish the generated implementation from the student's actual requirements decisions, reading, validation, changes and presentation activities.

Project Description

The application runs through main.py and serves a browser interface on the computer's loopback address. Most functionality uses the Python standard library. Separate modules handle resume parsing, database access, external data sources, ranking, AI requests and the HTTP application. SQLite stores normalized jobs, resume analysis, preferences and application progress. Each database operation uses a separate connection and parameterized SQL. This separation makes it possible to reason about persistence without mixing SQL statements throughout the interface code.

The input layer accepts pasted text and TXT, DOCX or text-based PDF resumes. It extracts known skill names with aliases and retains the relevant source lines as evidence. Two fixed public integrations collect jobs from Remotive and Arbeitnow. Additional jobs can enter through CSV, JSON, HTML containing JobPosting data, or manual entry. The revised project also includes a JobsDB research snapshot dated 28 September 2026. Its local converter validates 6,939 records, accepts 6,910 and records 29 rejections. The application imports 6,908 after excluding two jobs already marked expired. These frozen records are distinct from live public collection. The program retains source links and labels fictional classroom examples. External operations run in a bounded background task pool, allowing the interface to remain usable while a collection or model request is pending.

The ranking method combines weighted skill F1, TF-IDF cosine similarity, work experience, location preferences and educational requirements. Unknown conditions are omitted from the active weight denominator and exposed through a coverage indicator. The model does not provide the ranking score. DeepSeek instead supplies optional explanations and a separate revised resume draft. This design preserves a reproducible local baseline when the network or model service is unavailable.

Personal Contributions Requiring Verification

The available project record establishes that the user proposed a platform based on their resume, requested multiple recruitment sources and authorized a DeepSeek integration. It does not establish the user's personal implementation, testing or learning contributions. Before submission, the student should describe only work they actually perform and can explain. Suitable evidence could include a dated walkthrough of the ranking formula, a test they designed, a documented correction to a parsing rule, an evaluation using an appropriately anonymized resume, or a recorded explanation of an interface scenario.

This section must be replaced with the student's own account. It should identify the relevant file or feature, explain the decision made, describe the verification performed and state what changed as a result. If the contribution was reviewing AI output rather than writing code from scratch, that distinction should remain explicit. Group members should avoid submitting identical contribution statements. The purpose of the report is to show individual understanding and responsibility, not simply to summarize the capabilities of the finished program.

Challenges and Design Decisions

One observed challenge was misleadingly high scores for jobs that mention only generic skills. A candidate with English and communication experience can cover all listed skills of an unrelated writing job. The implementation was revised to use weighted F1 instead of requirement coverage alone, and English and communication receive lower weights. A regression test checks that a generic writing role does not outrank the relevant developer case. This change demonstrates why a technically valid formula still needs inspection against realistic examples.

Another challenge concerned the development computer's proxy configuration. DNS returned reserved Fake-IP addresses for legitimate external services. The fixed official integrations now use an explicit destination allowlist while retaining HTTPS certificate validation. Arbitrary webpage collection keeps its public-address checks. The program also refuses credential-bearing redirects. This solution supports the observed local environment without claiming that the application is a hardened public proxy or a production recruitment service.

External data quality remains uneven. Some listings omit salary or location, and remote work can still require residence in a specific country. The interface displays missing fields and keeps the original salary text instead of mixing currencies or payment periods. Network failures produce an error state, while local data and deterministic ranking remain available. These limitations are part of the intended demonstration rather than hidden exceptions.

Evaluation and Learning to Complete

The current implementation passes 71 local tests, including offline unit and HTTP integration tests covering parsing, malformed inputs, database persistence, duplicate records, AI response validation and local request boundaries. Real connectivity was separately checked against both job sources and DeepSeek using fictional resume content. A synthetic evaluation uses twelve authored jobs and four labelled relevant results. Its Precision at 4 is a narrow fixture result, not evidence of general hiring accuracy. The updated research benchmark ranks 6,910 accepted JobsDB records for a fictional resume. Three recorded local runs take 4.0517, 4.0059 and 4.0634 seconds, with a median of 4.0517 seconds on macOS with Python 3.8.10. This measures ranking runtime only. No human relevance labels were provided for the real snapshot, so real-data precision, recall and NDCG remain unmeasured. These records do not establish accuracy or fairness.

The student should run the tests personally, inspect at least one failure case and record the environment and observations. The final reflection should explain what the student actually learned about modules, regular expressions, transactions, background tasks or evaluation. It should also identify a specific improvement they would make next. Possible directions include broader skill coverage, an independently annotated dataset, better language handling or stricter candidate preferences. These are suggested learning activities, not claims that they have already happened.

Conclusion

JobCompass provides a working foundation for a Python course project with visible functionality and inspectable engineering decisions. Its strongest educational value comes from connecting file processing, data storage, information retrieval, network integration and testing in one application. Its limitations also create useful discussion about missing data, score interpretation and responsible AI assistance. Before submission, each group member must verify the program, understand the important code paths and replace this draft's contribution and learning passages with their own truthful account. The group must still supply student IDs and personally record the required 15-minute presentation. The classroom deck has five slides for five minutes.

Further Changes to Review Personally

The benchmark pipeline now records archive and normalized-content checksums, strips raw tracking and contact objects, and masks common contact text. Repeated imports preserve application notes. The public distribution contains a source ZIP and a selected normalized snapshot, with credentials and personal databases excluded. The student should inspect these decisions and explain any limitations they actually investigated.

A usability issue also revealed the difference between opening an HTML file and running a Python web application. The correct entry is `main.py` followed by `http://127.0.0.1:8765/`. The interface now uses relative asset paths and shows launch guidance in file mode. A final personal reflection could discuss this issue only after the author has reproduced it or reviewed the relevant code. This paragraph records the project change, not a claim that the student performed that verification.

References and AI Disclosure

Python documentation: <https://docs.python.org/3/>

DeepSeek JSON Output: https://api-docs.deepseek.com/guides/json_mode/

Remote API: <https://github.com/remotive-com/remote-jobs-api>

Arbeitsnow API: <https://www.arbeitsnow.com/blog/job-board-api>

Library and data-source details are listed in docs/REFERENCES.md. Codex assisted with development and this draft. DeepSeek is an optional runtime service.